

Code inventory — SPION_Quant

100 Python files, 22,950 lines, across two trees. Generated 2026-10-01. Each purpose is the first line of the module's own docstring, read from the file at build time.

1. SPION_Quant — 87 files

echoviewer — echo-time viewer and analysis engine

File	Purpose	Path
__init__.py	Echo-time viewer: the original DICOM viewer plus a TE axis.	echoviewer/__init__.py
__main__.py	Command-line entry point: ``python -m echoviewer [--sample NAME]``.	echoviewer/__main__.py
_bootstrap.py	Locate the original ``dicomview`` package and put it on ``sys.path``.	echoviewer/_bootstrap.py
dataset.py	Discovering samples and echo times inside a ``Volumes`` directory.	echoviewer/dataset.py
series.py	Cached access to one sample's volumes across echo times.	echoviewer/series.py
ui.py	Qt UI adding an echo-time axis to the original DICOM viewer.	echoviewer/ui.py
segmentation.py	Shape-agnostic segmentation, adapted from 2D slices to whole 3D volumes.	echoviewer/segmentation.py
preview.py	Rendering preview images of a segmentation, for human identification.	echoviewer/preview.py
export.py	Writing computed segmentations to disk, sparse and dense.	echoviewer/export.py
summary.py	Mean intensity per segmentation per echo time, exported to one workbook.	echoviewer/summary.py
fitting.py	Voxel-wise exponential decay fitting, with and without a baseline offset.	echoviewer/fitting.py
relaxometry.py	Driving the voxel-wise fit over both data sources.	echoviewer/relaxometry.py
model_selected.py	The AIC-selected parameter representation, and its I0-vs-rs visualisations.	echoviewer/model_selected.py
diagnostics.py	Diagnostic figures comparing Model A and Model B, and the class separation.	echoviewer/diagnostics.py

Top-level runners

File	Purpose	Path
run_echo_viewer.py	Standalone launcher for the echo-time DICOM viewer.	run_echo_viewer.py
run_diagnostics.py	Generate the Model A / Model B diagnostic figures and print the interpretation.	run_diagnostics.py
run_model_selected.py	Build the AIC-selected parameter representation and its I0-vs-rs figures.	run_model_selected.py
run_normalization.py	Compare I0 normalisation strategies before the classification stage.	run_normalization.py
run_save_normalized.py	Save the Strategy-B normalised data per segmentation, and plot the feature space.	run_save_normalized.py
run_high_rs_artifact.py	Diagnose the high-rs branch that Model B produces at high SPION concentration.	run_high_rs_artifact.py

data_treatment — normalisation and feature preparation

File	Purpose	Path
norm_comparison.py	Do secondary transformations of I0_normalized improve cross-phantom agreement?	data_treatment/norm_comparison.py
overlap_2d.py	2D (rs_selected, I0_normalized) overlap: by material, and by sample within material.	data_treatment/overlap_2d.py
overlap_prepost.py	Same material across phantoms: overlap before vs after Strategy-B normalisation.	data_treatment/overlap_prepost.py
residual_feature_space.py	Final feature-space check: rs_selected vs I0_residual, per phantom and combined.	data_treatment/residual_feature_space.py
verify_labellight.py	READ-ONLY verification of the proposed label-light normalisation.	data_treatment/verify_labellight.py
apply_labellight_normalization.py	Apply the verified label-light normalisation to training data and Pill2.	data_treatment/apply_labellight_normalization.py
normalize_pill2_volume.py	Normalise the complete Pill2 volume with the established pipeline.	data_treatment/normalize_pill2_volume.py
prepare_pill2_testing.py	Assemble the complete Pill2 volume for later classification.	data_treatment/prepare_pill2_testing.py
validate_fit.py	Validation run for the voxel-wise fitting, on one segmentation and one volume.	data_treatment/validate_fit.py
fit_report.py	Summarise the finished relaxometry run from the saved maps.	data_treatment/fit_report.py

data_treatment — feature diagnostics

File	Purpose	Path
b_parameter_diagnostic.py	Does B_selected add class separation? Read-only diagnostic, training vs Pill2.	data_treatment/b_parameter_diagnostic.py
b_normalization_transfer.py	READ-ONLY: does normalising B_selected make it phantom-invariant?	data_treatment/b_normalization_transfer.py
derived_feature_transfer.py	READ-ONLY: do dAIC, selected RMSE, dR2 and B_ratio transfer across phantoms?	data_treatment/derived_feature_transfer.py
shape_parameter_diagnostic.py	Do any dimensionless shape parameters transfer between phantoms?	data_treatment/shape_parameter_diagnostic.py
class_feature_plots.py	Per-sample rs x I0_residual scatter and per-class feature distributions. Read-only.	data_treatment/class_feature_plots.py
feature_space_train_vs_pill2.py	Training vs complete-Pill2 distributions in rs_selected x I0_residual. Read-only.	data_treatment/feature_space_train_vs_pill2.py
foreground_diagnosis.py	READ-ONLY: compare the Otsu foreground against the segmentation foreground for Pill2.	data_treatment/foreground_diagnosis.py
show_pill2_extent.py	Show what the 'full volume' prediction actually covers, in all three planes.	data_treatment/show_pill2_extent.py

data_treatment — classification

File	Purpose	Path
train_baseline_classifier.py	Baseline voxel-level concentration classifier, tested on Pill2 as a held-out phantom.	data_treatment/train_baseline_classifier.py
pill2_complete_segmentation_eval.py	Complete the Pill2 segmentation-foreground features and re-score the trained models.	data_treatment/pill2_complete_segmentation_eval.py
complete_dAIC_and_compare.py	Recover the missing Pill2 dAIC values and test dAIC as a classifier feature.	data_treatment/complete_dAIC_and_compare.py
classifier_benchmark.py	Controlled classifier benchmark on the fixed feature set rs_selected + I0_residual.	data_treatment/classifier_benchmark.py
classifier_benchmark_extension.py	Extend the classifier benchmark with LDA, QDA, GMM, hierarchical and ordinal.	data_treatment/classifier_benchmark_extension.py
classifier_benchmark_summary.py	Rebuild Classifiers/summary.csv from the saved per-classifier artifacts.	data_treatment/classifier_benchmark_summary.py

data_treatment — metrics

File	Purpose	Path
auc_metrics.py	One-vs-rest ROC AUC on Pill2, from saved probabilities only.	data_treatment/auc_metrics.py
iou_metrics.py	Per-class 3D IoU on Pill2 at a 0.7 confidence threshold, saved probabilities only.	data_treatment/iou_metrics.py
segmentation_metrics.py	Volume error, Brier, MCC, HD95 and ASSD on Pill2 from the saved predictions.	data_treatment/segmentation_metrics.py
organise_metrics.py	Reorganise the finished classifier metrics into Metrics///.	data_treatment/organise_metrics.py
write_metrics_summary_md.py	Write Metrics/summary.md.	data_treatment/write_metrics_summary_md.py
iou_corrected_labels.py	Pill2 IoU recomputed with Segmentation_4_0.25 relabelled as 0.2.	data_treatment/iou_corrected_labels.py
pill2_metrics_corrected_labels.py	Every Pill2 metric, recomputed with Segmentation_4_0.25 relabelled as 0.2.	data_treatment/pill2_metrics_corrected_labels.py

data_treatment — rankings and workbooks

File	Purpose	Path
rank_top3_models.py	Rank the top three classifiers for every metric already computed under Metrics/.	data_treatment/rank_top3_models.py
best_models_by_class.xlsx.py	Per-class top-3 classifier ranking as an Excel workbook, one sheet per class.	data_treatment/best_models_by_class.xlsx.py
rank_corrected_labels.py	Top-3 ranking and the per-class workbook, from the corrected-label metrics.	data_treatment/rank_corrected_labels.py
best_model_matrix_corrected_labels.py	One table: rows are metrics, columns are classes, cells name the best model.	data_treatment/best_model_matrix_corrected_labels.py
macro_ranking_sheets.py	Rank the classifiers on the macro averages, with and without tissue.	data_treatment/macro_ranking_sheets.py
class_ranking_sheet.py	Rank the classifiers on one class alone, as a new sheet in summary.xlsx.	data_treatment/class_ranking_sheet.py
weighted_summary_workbook.py	Support-weighted summary of every metric, one row per classifier.	data_treatment/weighted_summary_workbook.py
bump_chart_workbook.py	Rank-agreement (bump chart) data: how each classifier ranks under each metric.	data_treatment/bump_chart_workbook.py
results_workbooks.py	Organise the finished analysis into three Excel workbooks.	data_treatment/results_workbooks.py
results_workbooks_subsample.py	Rewrite the two rs-vs-I0 workbooks keeping a 10% subsample, for Excel's sake.	data_treatment/results_workbooks_subsample.py

data_treatment — figures and volumes

File	Purpose	Path
peak_slice_montage.py	Peak-probability slice per class, every method, coronal and sagittal.	data_treatment/peak_slice_montage.py
peak_slice_vector_svg.py	Fully vector version of the peak-slice montage: voxels as SVG rects.	data_treatment/peak_slice_vector_svg.py
gmm_probability_volume.py	Dense Pill2 class-probability volumes from the already-fitted GMM.	data_treatment/gmm_probability_volume.py
gmm_all_classes_volume.py	One Pill2 volume holding every GMM class, as a label map.	data_treatment/gmm_all_classes_volume.py
gmm_all_classes_overview_svg.py	Write all_classes_overview.svg from the saved all-class volume.	data_treatment/gmm_all_classes_overview_svg.py
all_models_probability_volumes.py	Pill2 probability volumes and a single all-class label volume, per classifier.	data_treatment/all_models_probability_volumes.py
all_models_axial_overviews.py	Axial companion to the sagittal all-class overviews, for every classifier.	data_treatment/all_models_axial_overviews.py
segmentation_overlays.py	Middle-slice overlays of every segmentation on the first-echo volume.	data_treatment/segmentation_overlays.py
rs_map_overview.py	rs-only overview for the training samples, peak-rs slice in each orientation.	data_treatment/rs_map_overview.py
echo_first_last_overview.py	First and last echo of one sample, three orthogonal views, rs-overview style.	data_treatment/echo_first_last_overview.py
coronal_montage.py	Coronal montage of the Pill2 full-volume prediction, every 10 slices.	data_treatment/coronal_montage.py
render_volume_3d.py	Isosurface rendering of one echo-time volume, shown from several viewpoints.	data_treatment/render_volume_3d.py
pill1_threshold_mask.py	Pill1 IO/rs threshold mask: IoU against segmentation 0.2, and two figures.	data_treatment/pill1_threshold_mask.py
make_colorbars.py	Standalone colourbar swatches at exact physical size, as SVG.	data_treatment/make_colorbars.py
__init__.py	Package marker (empty).	data_treatment/__init__.py

human — human and mouse intestine analysis

File	Purpose	Path
fit_and_normalise_human.py	IO and rs for the human/mouse intestine samples, with the phantom normalisation.	human/fit_and_normalise_human.py
tissue_anchored_normalisation.py	Tissue-anchored IO normalisation, applied identically to every dataset.	human/tissue_anchored_normalisation.py
rebuild_label_sheets.py	Rebuild the four label-derived sheets from the current Labels sheet.	human/rebuild_label_sheets.py

tests

File	Purpose	Path
conftest.py	Fixtures: a synthetic Volumes tree, plus opt-in access to the real dataset.	tests/conftest.py
test_echoviewer.py	Tests for echo-time discovery, volume loading and the cached series.	tests/test_echoviewer.py
test_fitting.py	Tests for the voxel-wise decay fitting.	tests/test_fitting.py
test_segmentation.py	Tests for the 3D segmentation and its integration into the viewer.	tests/test_segmentation.py
test_export.py	Tests for saving segmentations to disk, sparse and dense.	tests/test_export.py
test_summary.py	Tests for the mean-intensity workbook.	tests/test_summary.py

Other files

File	Purpose	Path
code_inventory_pdf.py	Build a PDF inventory of every Python file in both dicom_viewer trees.	code_inventory_pdf.py
config.py	Single place where every path and every frozen constant is defined.	config.py

2. dicom_viewer — the original viewer, 13 files

File	Purpose	Path
__init__.py	An interactive DICOM viewer with orthogonal views and SVG export.	dicomview/__init__.py
__main__.py	Command-line entry point: ``python -m dicomview [path]``.	dicomview/__main__.py
loader.py	Reading DICOM files and directories into :class:`~dicomview.volume.Volume`.	dicomview/loader.py
svg_export.py	Export the currently displayed plane as an SVG file.	dicomview/svg_export.py
ui.py	PyQt5 desktop UI for browsing DICOM volumes.	dicomview/ui.py
volume.py	Geometry-aware container for a stacked DICOM image volume.	dicomview/volume.py
windowing.py	Intensity normalisation and window/level handling.	dicomview/windowing.py
run_viewer.py	Standalone launcher for the DICOM viewer.	run_viewer.py
conftest.py	Shared fixtures: synthetic DICOM files covering the layouts the loader handles.	tests/conftest.py
test_loader.py	Tests for DICOM discovery, stacking, sorting and stack-axis classification.	tests/test_loader.py
test_svg_export.py	Tests for SVG export: valid XML, native-resolution image, true proportions.	tests/test_svg_export.py
test_volume.py	Tests for plane extraction, orientation and display aspect.	tests/test_volume.py
test_windowing.py	Tests for intensity normalisation and window/level behaviour.	tests/test_windowing.py